

Project 2 – Part 1: Implementing a Proxy Server

Authors: Georges Rizk (919388525), Shayan Khosh Keifi (923713063)

1. Output Screenshots

The tests we used were Ping, Pong, and abcd. These are the outputs for each separate program.

Client:

Sent to Proxy: ----- data = { "server_ip": "127.0.0.1" "server_port": 7000 "message": "Ping" } ----- Received from Proxy: ----- "Pong"	Sent to Proxy: ----- data = { "server_ip": "127.0.0.1" "server_port": 7000 "message": "Pong" } ----- Received from Proxy: ----- "Ping"	Sent to Proxy: ----- data = { "server_ip": "127.0.0.1" "server_port": 7000 "message": "abcd" } ----- Received from Proxy: ----- "dcba"
--	--	--

Proxy Server:

Received from Client: ----- data = { "server_ip": "127.0.0.1" "server_port": 7000 "message": "Ping" } ----- Sent to Server: ----- "Ping" ----- Received from Server: ----- "Pong" ----- Sent to Client: ----- "Pong"	Received from Client: ----- data = { "server_ip": "127.0.0.1" "server_port": 7000 "message": "Pong" } ----- Sent to Server: ----- "Pong" ----- Received from Server: ----- "Ping" ----- Sent to Client: ----- "Ping"	Received from Client: ----- data = { "server_ip": "127.0.0.1" "server_port": 7000 "message": "abcd" } ----- Sent to Server: ----- "abcd" ----- Received from Server: ----- "dcba" ----- Sent to Client: ----- "dcba"
--	--	--

Server:

Received from Proxy: ----- "Ping"	
Sent to Proxy: ----- "Pong"	Received from Proxy: ----- "abcd"
Received from Proxy: ----- "Pong"	
Sent to Proxy: ----- "Ping"	Sent to Proxy: ----- "dcba"

2. How to Run the Code

This project consists of three Python programs:

- `client_[GeorgesRizk]_[ShayanKhoshKeifi].py`
- `proxy_server_[GeorgesRizk]_[ShayanKhoshKeifi].py`
- `server_[GeorgesRizk]_[ShayanKhoshKeifi].py`

Run the system using three terminals.

Start the server:

```
python3 "server_[GeorgesRizk]_[ShayanKhoshKeifi].py"
```

Start the proxy server:

```
python3 "proxy_server_[GeorgesRizk]_[ShayanKhoshKeifi].py"
```

Run the client:

```
python3 "client_[GeorgesRizk]_[ShayanKhoshKeifi].py" Ping
```

```
python3 "client_[GeorgesRizk]_[ShayanKhoshKeifi].py" Pong
```

```
python3 "client_[GeorgesRizk]_[ShayanKhoshKeifi].py" abcd
```

3. Description of the Expected Result

This system demonstrates communication between a client and a server through a proxy server.

The client sends a JSON message containing the server IP, port, and a 4-character message to the proxy. The proxy checks whether the server IP is in a blocklist. If it is blocked, the proxy sends a 'Blocklist Error' back to the client. If the IP is allowed, the proxy forwards the message to the server. The server processes the message and returns a response. The proxy then forwards this response back to the client.

4. Initial Attempts and Challenges

During development we encountered several issues. TCP is a stream protocol, so a single `recv()` call may not receive the entire message. To solve this we implemented a helper function `'recv_all()'` that continuously reads data until the connection closes. We also had to ensure our output formatting matched the required format exactly. To accomplish this we implemented a helper function to print the JSON data block without commas.

5. High-Level Implementation Explanation

Client

The client accepts a 4-character string as a command-line argument. It creates a JSON object containing the server IP, port, and message and sends this JSON data to the proxy using a TCP socket. The client then waits for the response from the proxy and prints the returned result.

Proxy Server

The proxy server listens for incoming client connections. When it receives JSON data from the client, it extracts the server IP, port, and message. The proxy checks if the IP address is included in the blocklist. If the IP is blocked, it sends a 'Blocklist Error' message to the client. Otherwise, the proxy forwards the message to the server, receives the response, and forwards that response back to the client.

Server

The server listens for connections from the proxy server. When a message is received, the server processes it according to the assignment rules: 'Ping' returns 'Pong', 'Pong' returns 'Ping', and any other 4-character string is reversed. The computed response is then sent back to the proxy.

External Works

The following resources were used during the development of this project:

- Python socket documentation: This documentation was used to understand how to implement TCP and UDP in python.
- Python struct module documentation.
- RFC 1035 – Domain Names Implementation and Specification
- ChatGPT was used for conceptual explanations of networking concepts and debugging guidance.
- Lecture 10 – IP Protocol & Fragmentation
- Lecture 11 – Forwarding, ICMP, Naming
- Lecture 12 – ICMP, Naming, NAT
- Lecture 13 – Project 2 Concepts
- Lecture 14 – NAT and Routing
- Lecture 15 – Intra-Domain Routing
- These lectures provided background on networking concepts such as proxy servers, DNS naming, and host-to-IP address mapping. For example, the lecture discussing proxy servers explains how a proxy acts as an intermediary between a client and a server, which relates to the architecture implemented in this part.